ASSIGNMENT 1 – CO1 | TASK 11

Recommend and Defend the Final Design

Scenario: Namma Metro Smart-Card Fare System

Objective / explanation

The recommended design uses enum class, constexpr fare constants, integer paise and a private SmartCard balance. Fare calculation and balance updates are separate and easy to test.

C++ source code

```
#include &lt;iostream&gt;
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr long long FARES[] = {1000, 2000, 3000};
constexpr long long MIN_BALANCE = 5000;

class SmartCard {
    long long balance;
public:
    explicit SmartCard(long long initial) : balance(initial) {}
    bool deduct(long long fare) {
        if (fare &lt; 0 || balance &lt; fare || balance - fare &lt; MIN_BALANCE)
            return false;
        balance -= fare;
        return true;
    }
    long long getBalance() const { return balance; }
};
int main() {
    SmartCard card(15000);
    cout &lt;&lt; "Trip 1 accepted: " &lt;&lt; card.deduct(3000) &lt;&lt; '\n';
    cout &lt;&lt; "Trip 2 accepted: " &lt;&lt; card.deduct(7000) &lt;&lt; '\n';
    cout &lt;&lt; "Final balance paise: " &lt;&lt; card.getBalance() &lt;&lt; '\n';
}
```

Compilation

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o program
```

Actual captured program output

```
Trip 1 accepted: 1
Trip 2 accepted: 1
Final balance paise: 5000
```

Output screenshot

Analysis / complexity

Both trips are accepted: after deducting 3000 paise, the balance is 12000; deducting 7000 leaves exactly 5000 paise, which satisfies the minimum-balance rule. The result illustrates why the condition must be balance - fare >= 5000. Both operations take O(1) time.

Evidence note: Output shown is captured by executing this source in the generation environment. Recompile and run on your own system, and use your own screenshots if your instructor requires evidence from your computer.